Software Requirement specification

A Software Requirements Specification (SRS) for ConnectomeChronicle defines the functional and non-functional requirements of the proposed temporal, federated, and distributed database system for managing heterogeneous neural connectome datasets. It specifies the system's requirements for dataset ingestion and schema integration, source-to-canonical neuron mapping, temporal and provenance-aware data management, ConnectomeDiff, distributed storage and query processing, and the supporting AI/ML components. The SRS also describes the expected interactions between researchers and the system through its database, APIs, and visualization interface, providing a clear foundation for system design, implementation, testing, and evaluation while reducing ambiguity and redesign during later development stages.

Hardware Requirements

- **Client-Side:** A desktop or laptop system running Windows, Linux, or macOS with a modern web browser to access the ConnectomeChronicle dashboard.
- **Server-Side / Development:**
  A multi-core processor with at least 16 GB RAM recommended for connectome ingestion, graph processing, and database operations. Adequate SSD/local or cloud storage for large connectome datasets, PostgreSQL databases, MongoDB collections, and intermediate ETL outputs. A dedicated GPU is optional but beneficial for GraphSAGE training and large-scale graph-embedding computation.

Software Requirements

- **Client-Side:** A modern web browser such as Google Chrome, Mozilla Firefox, or Safari with JavaScript support. Web interface developed using React and Vite for visualization and interaction with the database and analytical services.
- **Server-Side / Development:**
  - **Relational Database:** PostgreSQL as the authoritative database for canonical entities, releases, source neurons, mappings, connections, annotations, temporal records, provenance, and partition metadata.
  - **NoSQL Database:** MongoDB for heterogeneous and source-specific metadata, morphology information, experimental attributes, and semi-structured annotations.
  - **Backend:** Python with FastAPI for RESTful API services and database/query orchestration.
  - **Data Processing:** Pandas and NumPy for ETL, normalization, transformation, and feature preparation; NetworkX for connectome graph construction and analysis.
  - **Machine Learning:** PyTorch Geometric/GraphSAGE and graph embeddings for

cross-dataset neuron candidate generation; Scikit-learn Isolation Forest for detecting statistically unusual changes identified by ConnectomeDiff.

- **Distributed Environment:** Docker and Docker Compose for deploying multiple PostgreSQL instances/shards and evaluating distributed storage and query-processing strategies.

Functional Requirements

1) Dataset Ingestion and Schema Integration

• The system must ingest heterogeneous connectome datasets through source-specific ETL adapters.
• The ingestion pipeline must preserve original source identifiers, metadata, files, and source records to maintain source identity and traceability.
• The system must normalize source-specific schemas into a canonical connectome representation containing entities such as neurons, neuron types, brain regions, connections, and releases.
• The architecture must allow additional connectome datasets to be integrated without redesigning the core canonical schema.

2) Cross-Dataset Entity Resolution

• The system must establish correspondences between source-specific neurons and canonical neurons across different datasets.
• Graph structure, connectivity patterns, metadata, and biological attributes must be used as features for candidate generation.
• Graph embeddings and GraphSAGE must assist in ranking potential cross-dataset neuron correspondences.
• The system must store the mapping method, confidence score, supporting evidence, and mapping timestamp.
• Ambiguous and unresolved correspondences must be retained explicitly rather than being treated as definitive mappings.

3) Temporal and Version-Aware Data Management

• The system must maintain multiple dataset releases and versions without overwriting historical information.
• Neuron, connection, and annotation records must support historical state management through valid-time and transaction-time intervals, where applicable.

• The system must support reconstruction of the connectome state corresponding to a particular release or point in time.
• Historical queries must distinguish between the period in which a fact was valid and the period in which it was recorded in the database.

4) Provenance and Data Lineage

• The system must maintain provenance for source and derived records throughout the integration pipeline.
• Provenance records must capture the source dataset, release, source file, source record, row/location information, transformation performed, pipeline version, and ingestion timestamp.
• Entity mappings, annotations, and detected changes must remain traceable to their underlying source information and processing steps.
• The provenance layer must support reproducibility and verification of integrated connectome data.

5) Connectome Difference and Change Analysis

• The system must compare different releases or historical states and identify added, deleted, and modified neurons, connections, and annotations.
• ConnectomeDiff must represent these changes using the canonical schema and associate them with relevant releases, temporal information, and provenance.
• Connectivity changes such as variations in edge weights, synapse counts, and local connectivity must be available for analysis.
• Isolation Forest must analyse derived change features and flag statistically unusual changes for further investigation without treating them as biological errors automatically.

6) Distributed Storage and Query Processing

• The system must support deployment of the relational database across multiple PostgreSQL shards using Docker/Docker Compose.
• The system must implement and compare identifier-based, anatomical-region-based, and connectivity-community-based partitioning strategies.
• The query layer must determine the relevant shard or shards for a request and support distributed query execution where cross-shard data is required.
• The system must evaluate partitioning strategies using query latency, cross-shard communication, data transfer, load balance, and distributed join overhead.

7) Query Processing, API and Visualization

• The system must expose RESTful FastAPI endpoints for querying datasets, releases, neurons, connections, mappings, annotations, provenance, changes, and historical states.
• The query layer must support metadata queries, historical reconstruction, cross-dataset correspondence queries, and ConnectomeDiff operations.
• The React-based dashboard must provide visualizations of connectome structure, neuron mappings, release differences, provenance, and distributed storage information.
• Query results must remain consistent with the authoritative PostgreSQL state and associated MongoDB metadata.

8) Database Integrity, Consistency and Reproducibility

• PostgreSQL must remain the authoritative relational source of integrated connectome state, while MongoDB must provide flexible storage for heterogeneous source-specific information.
• The system must enforce primary-key, foreign-key, uniqueness, domain, and referential constraints across related entities.
• Transactions and indexing must be used to maintain consistency and support efficient query execution over highly interconnected records.
• Database, ETL, ML, and derived analytical outputs must be associated with appropriate dataset versions, pipeline configurations, and model versions to ensure reproducibility.